FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate



BÀI 4. THỰC HIỆN DỰ ÁN 2

Giảng viên:

https://caodang.fpt.edu.vn

MỤC TIÊU

- Tìm hiểu lớp tiện ích: FileUtil.
- Xác thực: Authentication
- Phân quyền: Authorization
- Xây dựng chức năng đăng nhập
- Xây dựng chức năng quản lý danh mục loại đồ uống(CRUD)

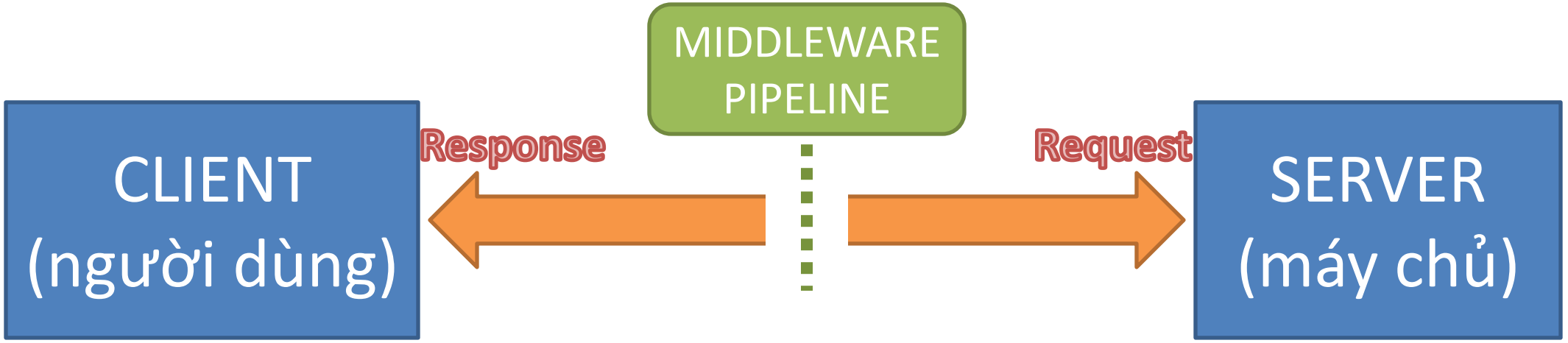




XÁC THỰC VÀ PHÂN QUYỀN

MIDDLEWARE PIPELINE

- Là những xử lý trung gian giữa các Request và Response.
- Giúp kiểm tra và xác thực thông tin người dùng trước khi dữ liệu được gửi đến Máy chủ



MIDDLEWARE PIPELINE

Cài đặt Program.cs:

```
builder.Services.AddTransient<UserDAL>();
// Xác thực: dựa trên cookie
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Login/Login";
        options.AccessDeniedPath = "/Home/AccessDenied";
        //Xác định thời gian tồn tại của cookie là 24 giờ
        options.ExpireTimeSpan = TimeSpan.FromHours(24);
    });
// Phân quyền: các policy cho vai trò
builder.Services.AddAuthorization(options =>
{
    // policy cho phép cả Employee và Manager (chấp nhận một trong hai)
    options.AddPolicy(
                "EmployeeOrManager", policy => policy.RequireRole("Employee", "Manager"));
    // policy chỉ cho phép Manager
    options.AddPolicy("ManagerOnly", policy => policy.RequireRole("Manager"));
});
var app = builder.Build();
```

MIDDLEWARE PIPELINE

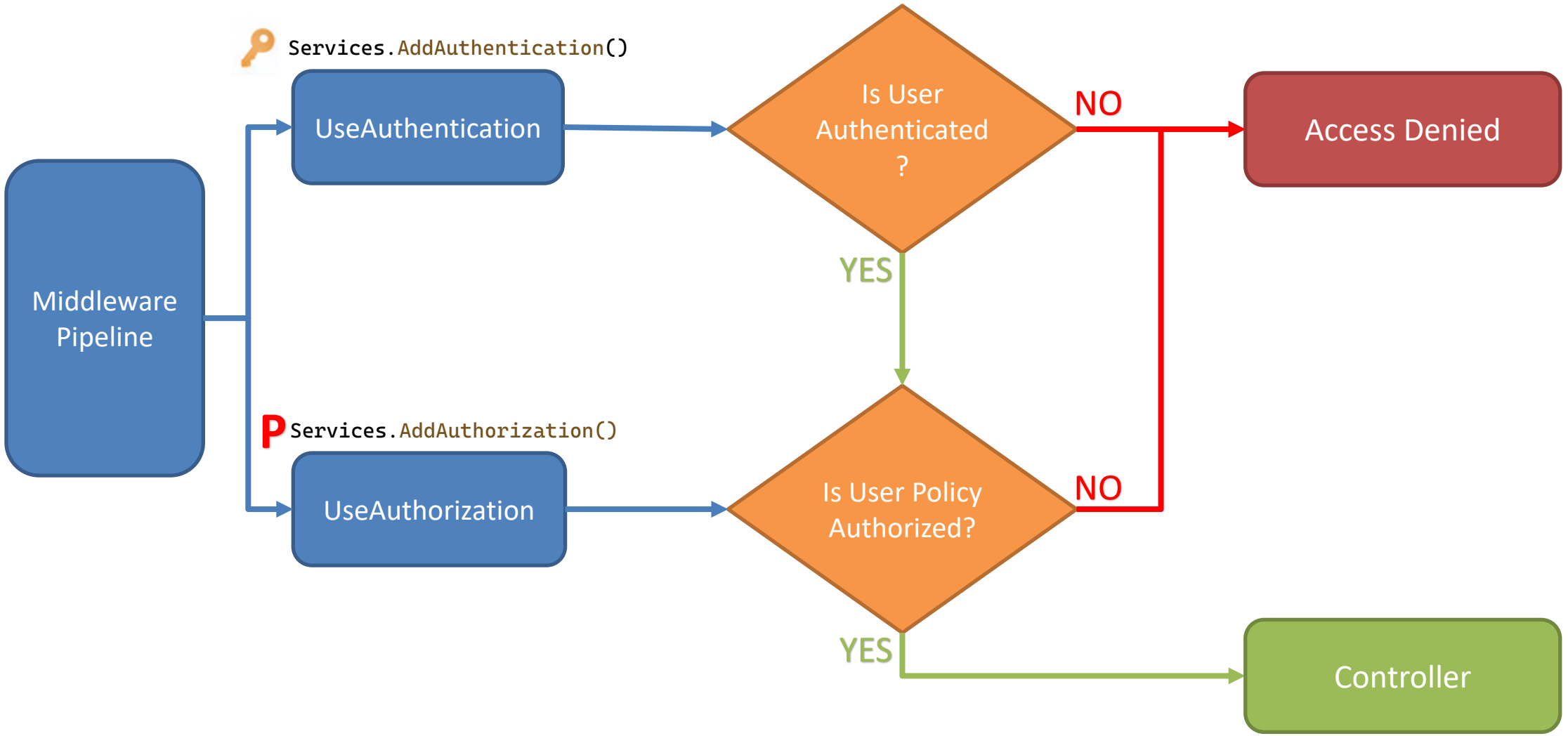
- Cài đặt Program.cs:

```
// Cấu hình pipeline xử lý HTTP.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
// QUAN TRỌNG: Xác thực phải được gọi trước Phân quyền
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

MIDDLEWARE PIPELINE

- Tài khoản sau khi đăng nhập sẽ được lưu trữ lại cookie 24 giờ.
- Authentication giúp xác định tài khoản đã đăng nhập hay chưa.
- Authorization giúp kiểm tra tài khoản đang là quyền gì, có thể truy cập đến đường dẫn hiện tại hay không.

MIDDLEWARE PIPELINE



MIDDLEWARE PIPELINE

- Tài khoản sau khi đăng nhập sẽ được lưu trữ lại cookie 24 giờ.
- Authentication giúp xác định tài khoản đã đăng nhập hay chưa.
- Authorization giúp kiểm tra tài khoản đang là quyền gì, có thể truy cập đến đường dẫn hiện tại hay không.

MIDDLEWARE PIPELINE

Các thẻ định nghĩa phân quyền cho routing: các thẻ phân quyền này sẽ áp dụng vào các phương thức ở Controller

- [AllowAnonymous]: Truy cập tự do
- [Authorize]: Cần đăng nhập
- [Authorize(Policy = "EmployeeOrManager")]:

Cho phép cả tài khoản Nhân viên và Quản lý truy cập

- [Authorize(Policy = "ManagerOnly")]:

Chỉ cho phép tài khoản Quản lý truy cập

FileUtil

- Lớp này chứa các phương thức tiện ích hỗ trợ upload file hoặc xóa file.

PHƯƠNG THỨC	CÔNG DỤNG
Upload(IFormFile file, string webRootPath)	Thực hiện việc upload file
Delete(string fileName, string webRootPath)	Xóa file trong thư mục upload

FileUtil

- Xây dựng phương thức upload.

```
/// <summary> Uploads an IFormFile to the web root /uploads and returns the new  ...
0 references
public static string Upload(IFormFile file, string webRootPath)
{
    if (file is null) throw new ArgumentNullException(nameof(file));
    if (string.IsNullOrWhiteSpace(webRootPath)) throw new ArgumentNullException(nameof(webRootPath));

    var uploadsDir = Path.Combine(webRootPath, UploadsFolderName);
    Directory.CreateDirectory(uploadsDir);

    var ext = Path.GetExtension(file.FileName) ?? string.Empty;
    var timestamp = DateTime.UtcNow.ToString("yyyyMMddHHmmssfff");
    var shortGuid = Guid.NewGuid().ToString("N").Substring(0, 8);
    var safeFileName = $"{timestamp}_{shortGuid}{ext}";

    var destPath = Path.Combine(uploadsDir, safeFileName);

    using (var stream = new FileStream(destPath, FileMode.Create))
    {
        file.CopyTo(stream);
    }

    return safeFileName;
}
```

- Xây dựng phương thức delete.

```
/// <summary> Deletes a file from the web root /uploads folder by file name.
0 references
public static bool Delete(string fileName, string webRootPath)
{
    if (string.IsNullOrWhiteSpace(fileName)) throw new ArgumentNullException(nameof(fileName));
    if (string.IsNullOrWhiteSpace(webRootPath)) throw new ArgumentNullException(nameof(webRootPath));

    try
    {
        var filePath = Path.Combine(webRootPath, UploadsFolderName, fileName);
        if (!File.Exists(filePath))
            return false;

        File.Delete(filePath);
        return true;
    }
    catch
    {
        return false;
    }
}
```

FileController

- Controller này giúp thực hiện upload hoặc xóa hình ảnh.

```
[Authorize]
1 reference
public class FileController : Controller
{
    private readonly IWebHostEnvironment _env;

    0 references
    public FileController(IWebHostEnvironment env)
    {
        _env = env;
    }

    // Upload image (Manager only)
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Upload(IFormFile file)...

    // Delete image by file name (Manager only)
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Delete(string fileName)...
}
```

FileController

- Phương thức Upload từ HttpPost.

```
public IActionResult Upload(IFormFile file)
{
    if (file == null || file.Length == 0)
        return BadRequest(new { success = false, error = "No file provided." });

    try
    {
        var savedFileName = FileUtil.Upload(file, _env.WebRootPath);
        var url = Url.Content($"~/uploads/{savedFileName}");
        return Ok(new { success = true, fileName = savedFileName, url });
    }
    catch (System.Exception ex)
    {
        return BadRequest(new { success = false, error = "Upload failed.",
detail = ex.Message });
    }
}
```

Phương thức Delete từ HttpPost.

```
if (string.IsNullOrWhiteSpace(fileName))
    return BadRequest(new { success = false, error = "File name is required."
});

try
{
    var ok = FileUtil.Delete(fileName, _env.WebRootPath);
    if (!ok)
        return BadRequest(new { success = false, error = "File not found or
delete failed." });

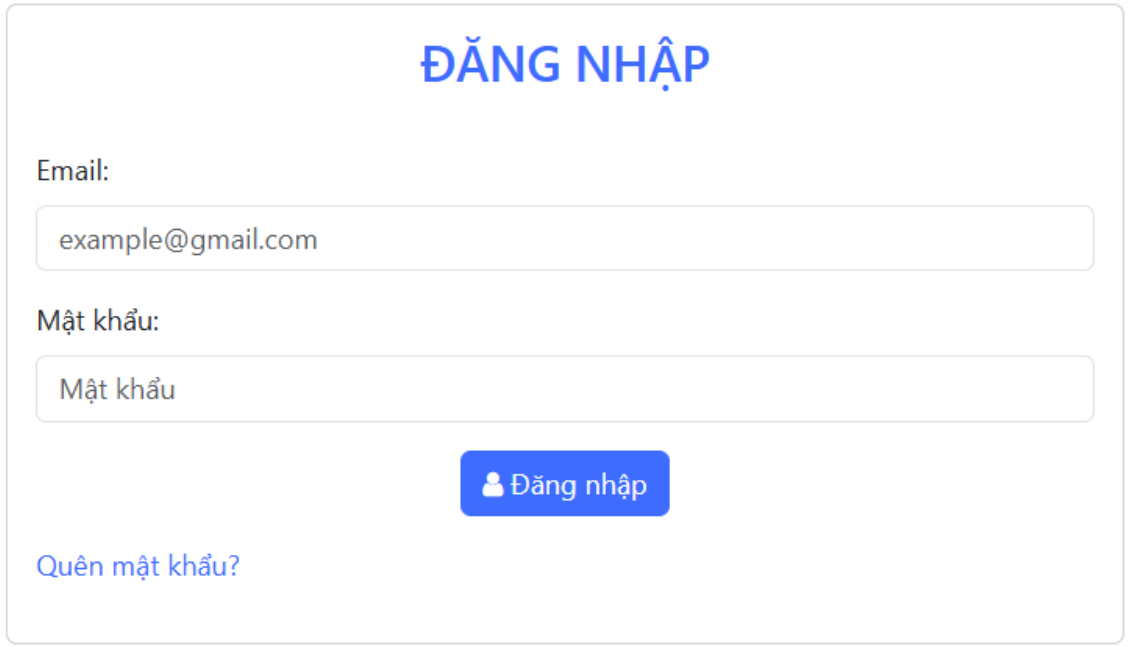
    return Ok(new { success = true });
}
catch (System.Exception ex)
{
    return BadRequest(new { success = false, error = "Delete failed.", detail =
ex.Message });
}
```



XÂY DỰNG CHỨC NĂNG ĐĂNG NHẬP

ĐĂNG NHẬP

- Đăng nhập là các chức năng được sử dụng với mọi người dùng (quản lý hay nhân viên).
- Để sử dụng website PolyCoffee thì mọi nhân viên buộc phải đăng nhập.
- Giao diện



ĐĂNG NHẬP

- Xây dựng phương thức Authenticate(string email, string password) trong UserDAL.

```
public static UserDAL? Authenticate(string email, string password)
{
    string sql = "SELECT id, email, password, full_name, phone,
role, active FROM users WHERE email = @1 AND password = @2 AND
active = 1";
    var dt = DBUtil.queryDataTable(sql, new List<object> { email,
password });
    if (dt.Rows.Count == 0) return null;
    return FromDataRow(dt.Rows[0]);
}
```

ĐĂNG NHẬP

- Cài đặt mã cho LoginController
- Phương thức Login ở HttpGet dùng để hiển thị giao diện

```
[HttpGet]
[AllowAnonymous]
public IActionResult Login(string returnUrl = null)
{
    ViewData["Title"] = "Login";
    var model = new LoginViewModel
    {
        ReturnUrl = returnUrl ?? Url.Content("~/")
    };
    return View("Views/Account/Login.cshtml", model);
}
```

ĐĂNG NHẬP

- Cài đặt mã cho LoginController
- Phương thức Login ở HttpPost dùng để xử lý đăng nhập

```
[HttpPost]
[ValidateAntiForgeryToken]
[AllowAnonymous]
public async Task<IActionResult> Login(LoginViewModel model, string returnUrl = null)
{
    model.ReturnUrl = returnUrl ?? model.ReturnUrl ?? Url.Content("~/");
    if (!ModelState.IsValid)
    {
        return View("Views/Account/Login.cshtml", model);
    }
    var user = UserDAL.Authenticate(model.Email ?? string.Empty, model.Password ?? string.Empty);
    if (user != null) {
        ...
    }
    ModelState.AddModelError(string.Empty, "Invalid email or password.");
    return View("Views/Account/Login.cshtml", model);
}
```

ĐĂNG NHẬP

```
if (user != null)
{
    var roleName = user.Role ? "Manager" : "Employee";
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Name, user.FullName ?? user.Email ?? string.Empty),
        new Claim(ClaimTypes.Email, user.Email ?? string.Empty),
        new Claim(ClaimTypes.Role, roleName)
    };
    var identity = new ClaimsIdentity(claims,
        CookieAuthenticationDefaults.AuthenticationScheme);
    var principal = new ClaimsPrincipal(identity);
    var authProperties = new AuthenticationProperties
    {
        IsPersistent = model.RememberMe
    };
    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme, principal, authProperties);
    return LocalRedirect(model.ReturnUrl);
}
```

ĐĂNG NHẬP

- Sau khi thực hiện đăng nhập thành công thì tài khoản đã đăng nhập sẽ được lưu trong Cookie.
- Khi có bất cứ xử lý nào gọi đến Middleware Pipeline sẽ chặn lại và kiểm tra quyền trong Cookie tương ứng.

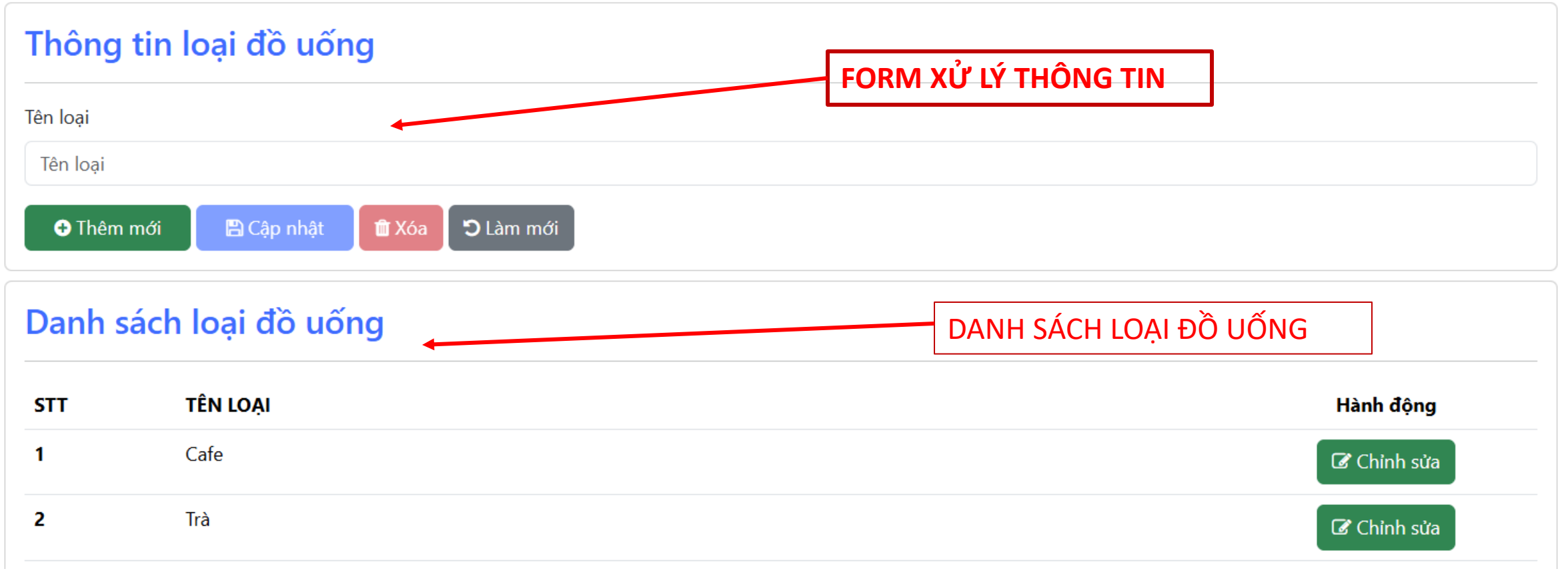


XÂY DỰNG CHỨC NĂNG QUẢN LÝ LOẠI ĐỒ UỐNG

FPT Education
FPT POLYTECHNIC

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Quản lý(Xem danh sách, thêm mới, sửa, xóa) loại đồ uống.
- Giao diện:



QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt các phương thức cần thiết trong CategoryDAL

```
public class CategoryDAL
{
    15 references
    public int Id { get; set; }
    16 references
    public string? Name { get; set; }
    9 references
    public bool Active { get; set; }

    2 references
    public CategoryDAL() { }

    2 references
    private static CategoryDAL FromDataRow(DataRow row)...

    1 reference
    public static List<CategoryDAL> GetAll()...

    3 references
    public static CategoryDAL? GetById(int id)...

    1 reference
    public static int Insert(CategoryDAL category)...

    1 reference
    public static bool Update(CategoryDAL category)...

    1 reference
    public static bool Delete(int id)...
}
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho CategoryController
- Tài khoản nhân viên chỉ có quyền xem danh sách và chi tiết của Loại đồ uống, không thể thêm, sửa, xóa.

```
// Read: Employee or Manager
[Authorize(Policy = "EmployeeOrManager")]
3 references
public IActionResult Index()...

[Authorize(Policy = "EmployeeOrManager")]
0 references
public IActionResult Details(int id)...

// Create: Manager only
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult Create()...

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult Create(CategoryDAL category)...
```

```
// Edit: Manager only
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult Edit(int id)...

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult Edit(int id, CategoryDAL category)...

// Delete: Manager only
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult Delete(int id)...

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
0 references
public IActionResult DeleteConfirmed(int id)...
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho phương thức Index và Details

```
[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Index()
{
    var list = CategoryDAL.GetAll();
    return View(list);
}

[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Details(int id)
{
    var category = CategoryDAL.GetById(id);
    if (category == null) return NotFound();
    return View(category);
}
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho phương thức Create và Update ở HttpGet
- 2 phương thức này giúp hiển thị giao diện khi thêm hoặc chỉnh sửa

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create()
{
    return View(new CategoryDAL());
}

[Authorize(Policy = "ManagerOnly")]
public IActionResult Edit(int id)
{
    var category = CategoryDAL.GetById(id);
    if (category == null) return NotFound();
    return View(category);
}
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho phương thức Create ở HttpPost
- Xử lý thêm mới dữ liệu từ Form

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create(CategoryDAL category)
{
    if (category == null) return BadRequest();
    if (string.IsNullOrWhiteSpace(category.Name))
        ModelState.AddModelError(nameof(category.Name), "Name is required.");
    if (!ModelState.IsValid)
        return View(category);
    CategoryDAL.Insert(category);
    return RedirectToAction(nameof(Index));
}
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho phương thức Edit ở HttpPost
- Xử lý chỉnh sửa lại dữ liệu từ Form

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult Edit(int id, CategoryDAL category)
{
    if (category == null) return BadRequest();
    if (id != category.Id) return BadRequest();
    if (string.IsNullOrWhiteSpace(category.Name))
        ModelState.AddModelError(nameof(category.Name), "Name is required.");
    if (!ModelState.IsValid)
        return View(category);
    var ok = CategoryDAL.Update(category);
    if (!ok) ModelState.AddModelError(string.Empty, "Update failed.");
    if (!ModelState.IsValid) return View(category);
    return RedirectToAction(nameof(Index));
}
```

QUẢN LÝ LOẠI ĐỒ UỐNG(CATEGORIES)

- Cài đặt mã cho phương thức Delete và DeleteConfirmed

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Delete(int id)
{
    var category = CategoryDAL.GetById(id);
    if (category == null) return NotFound();
    return View(category);
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult DeleteConfirmed(int id)
{
    var ok = CategoryDAL.Delete(id);
    if (!ok) return BadRequest();
    return RedirectToAction(nameof(Index));
}
```

Summary

- Tìm hiểu lớp tiện ích: FileUtil.
- Xác thực: Authentication
- Phân quyền: Authorization
- Xây dựng chức năng đăng nhập
- Xây dựng chức năng quản lý danh mục loại đồ uống(CRUD)




FPT Education
FPT POLYTECHNIC
Thank you